

Production Mode Workflow Compiler and Execution Optimization Plan

Objective

Add a sidebar checkbox named:

Simulate Production

When disabled, all existing workflow submission, execution, output collection, logging, previews, and return behavior must remain unchanged.

When enabled, submit and execute a production-compiled copy of the workflow that:

1. Keeps only nodes required by explicitly selected production outputs.
2. Applies nodes marked Bypass in Production using ComfyUI's native bypass serialization behavior.
3. Removes all dead branches, including preview, comparer, debug-save, and unused output branches.
4. Disables sampler latent previews before sampling.
5. Caches the production execution plan by graph topology.
6. Executes only the retained output nodes.
7. Replaces compatible final image-output nodes with a remote in-memory output sink.
8. Avoids PNG-to-disk-to-read-to-convert round trips where supported.
9. Converts each unique fallback file only once.
10. Returns only selected production outputs.
11. Suppresses noisy execution logs while preserving errors and compact timing summaries.
12. Throttles progress events.
13. Reuses validation results for unchanged production graph structures.
14. Reports duplicate and unnecessary workflow work without automatically rewriting complex compute nodes.

Do not install or depend on ComfyUI-to-Python-Extension.

Do not replace ComfyUI's PromptExecutor.

Do not change model loading, model caches, restore-background UNET, direct CLIP warmup, VAE loading policy, CacheDiT, sampler settings, scheduler settings, step count, precision, output dimensions, or image quality.

Fixed user-facing behavior

Sidebar control

Add one checkbox to the existing comfyui-modal sidebar:

Simulate Production

Default:

- Off for workflows that have never stored a value.
- The value is saved in the workflow's ComfyUI metadata so each workflow remembers its setting.
- Do not create a separate settings page.
- Do not create a second sidebar panel.

Store the workflow-level value as:

```
{
  "extra": {
    "comfymodal": {
      "production_mode_enabled": true
    }
  }
}
```

Use the existing workflow extra-metadata mechanism already used by the frontend. Do not inject this metadata into API-prompt node specifications.

Node context-menu actions

Add these actions to every appropriate node's existing context menu:

For output-capable nodes:

- Mark as Production Output
- Unmark Production Output

For all ordinary nodes:

- Bypass in Production
- Do Not Bypass in Production

Store these flags in the frontend node properties:

```
node.properties.comfymodal_production_output = true
node.properties.comfymodal_bypass_in_production = true
```

Do not change the node's normal ComfyUI mode when saving or editing the workflow.

Show a small visible badge on marked nodes:

- Production Output: green badge labeled PROD OUT
- Bypass in Production: amber badge labeled PROD BYPASS

Do not change the node title.

First enable behavior

When Simulate Production is enabled and the workflow has zero nodes marked as Production Output:

1. Find all output-capable nodes in the current graph using ComfyUI's registered node metadata.
2. Open a selection modal listing those output nodes.
3. Show:
 4. node ID
 5. node title
 6. node class type
7. Require the user to select at least one output.
8. Save selected nodes as Production Outputs.
9. Cancel enabling Production Mode if the modal is cancelled.

Do not use heuristics to silently guess the production output.

After at least one Production Output has been configured, future runs require no modal.

Sidebar production summary

Under the checkbox, display a compact read-only summary:

```
Production plan
Kept: 24 nodes
Removed: 7 nodes
Bypassed: 1 node
Outputs: 1
Sampler previews: disabled
Direct outputs: 1
```

If the plan is invalid, show the error instead and disable queueing in Production Mode.

Data contract

Extend the existing `modal_options` payload.

Add:

```
{
  "production": {
    "enabled": true,
    "schema_version": 1,
    "output_node_ids": ["92"],
    "bypass_node_ids": ["47"],
    "disable_sampler_previews": true,
    "quiet_execution_logs": true,
    "progress_min_interval_ms": 500,
    "strict_output_collection": true,
    "direct_output_sink": true,
    "metadata_mode": "none"
  }
}
```

Rules:

- Node IDs are always serialized as strings.
- Sort `output_node_ids` and `bypass_node_ids` numerically where possible, otherwise lexicographically.
- `schema_version` must equal `1`.
- Production defaults are exactly:
 - `disable_sampler_previews`: true
 - `quiet_execution_logs`: true
 - `progress_min_interval_ms`: 500
 - `strict_output_collection`: true
 - `direct_output_sink`: true
 - `metadata_mode`: none
- If `production` is missing or `enabled` is false, use existing behavior.
- Reject unknown production schema versions.
- Do not accept production output IDs from arbitrary remote callers without validating that those IDs exist in the submitted workflow.

New shared module

Create:

```
production_workflow.py
```

This module must contain pure workflow-transformation code with no Modal, Torch, ComfyUI server, filesystem, or GPU dependencies.

Add `production_workflow` to `_COMFYMODAL_LOCAL_PYTHON_SOURCES` in `comfyapp.py` so the same implementation is available locally and remotely.

Expose these public functions:

```
normalize_production_options(modal_options: dict | None) -> dict
build_production_topology_hash(workflow: dict, production: dict) -> str
compile_production_workflow(
    workflow: dict,
    production: dict,
    *,
    allow_direct_output_rewrite: bool,
) -> tuple[dict, dict]
analyze_duplicate_work(workflow: dict) -> dict
```

Exact production compilation algorithm

Input normalization

`normalize_production_options()` must:

1. Return `{"enabled": False}` when no production options exist.
2. Require a dict when production options exist.
3. Require `schema_version == 1`.
4. Normalize all node IDs to strings.
5. Remove duplicate node IDs.
6. Sort IDs deterministically.
7. Apply the fixed defaults listed above.
8. Reject an empty `output_node_ids` list when enabled.
9. Reject a node appearing in both `output_node_ids` and `bypass_node_ids`.

Connection detection

Treat an API-prompt input as a node connection only when all conditions are true:

```
isinstance(value, (list, tuple))
len(value) == 2
str(value[0]) in workflow
isinstance(value[1], int)
```

Do not interpret arbitrary two-item lists as connections unless the first item resolves to a real workflow node.

Dead-branch traversal

`compile_production_workflow()` must:

1. Deep-copy neither the full graph nor tensor data.
2. Normalize the workflow's node IDs to strings in a shallow normalized map.
3. Verify every Production Output node exists.
4. Verify every Production Output node is not marked for bypass.
5. Start a reverse graph traversal from all `output_node_ids`.
6. For every retained node:
7. inspect every input
8. identify node connections
9. add each connected source node to the traversal
10. Continue until no new ancestors exist.
11. Remove every node not reached.
12. Preserve the original node insertion order in the returned dict.
13. Preserve current scalar input values.
14. Preserve node class types and all inputs exactly for retained nodes, except direct-output rewrites described below.
15. Never add upstream nodes that are not ancestors of a selected Production Output.

Return a report containing:

```
{
  "enabled": True,
  "schema_version": 1,
  "original_node_count": int,
  "compiled_node_count": int,
  "removed_node_count": int,
  "kept_node_ids": list[str],
  "removed_node_ids": list[str],
  "output_node_ids": list[str],
  "bypass_node_ids": list[str],
  "direct_output_rewritten_node_ids": list[str],
  "topology_hash": str,
  "cache_hit": bool,
  "duplicate_analysis": dict,
}
```

Bypass behavior

Do not implement generic backend rewiring for Bypass in Production.

Use ComfyUI's native frontend bypass serialization.

For a Production Mode queue submission:

1. Read all nodes marked `comfy_modal_bypass_in_production`.
2. Save each node's original `mode`.
3. Temporarily set those nodes to LiteGraph bypass mode `4`.
4. Invoke the same existing API-prompt serialization path currently used for a normal Modal submission.
5. Restore every original node mode in a `finally` block.
6. Continue production compilation using the resulting serialized API prompt.

The temporary mode change must never be written into the saved workflow.

Do not queue another prompt while temporary modes are applied.

After serialization, verify that every bypass-marked node is absent from the serialized API prompt. If any marked node remains, stop with:

```
Production bypass failed for node <id> (<class_type>). ComfyUI could not
serialize this node as a native bypass.
```

Do not guess an input/output rewiring for unsupported nodes.

The backend receives `bypass_node_ids` only for validation and reporting. It must not attempt a second rewiring pass.

Inline bypass consequence

Because native bypass serialization runs before dead-branch elimination:

- Inline development nodes can be bypassed.
- Their downstream production path remains reachable.
- Dead branches created by the bypass are then removed.
- Output/debug branches not selected as production roots are removed automatically.

Topology cache

Create an in-memory cache both locally and in the Modal container.

Do not persist this cache to the model volume.

Do not call `vol.commit()`.

Cache only the retained node-ID plan and transformation report, not stale scalar values.

Topology hash

`build_production_topology_hash()` must include:

- compiler schema version
- every node ID
- every node class type
- every input key
- every connection source node ID
- every connection source output slot
- sorted Production Output IDs
- sorted Bypass IDs
- whether direct output rewriting is enabled

For literal scalar inputs, include only:

- the input key
- the literal value's type name

Do not include literal values in the production topology hash.

This means prompt text, seeds, dimensions, strengths, and similar dynamic values do not force recompilation, while graph connection changes do.

Cache behavior

Use a bounded 32-entry LRU cache.

On cache hit:

1. Retrieve the retained node-ID set and direct-output rewrite instructions.
2. Build a new compiled workflow from the current request's node specifications.
3. Never return the cached workflow dict itself.
4. Preserve current prompt, seed, dimensions, input-image names, denoise values, and all other scalar values.

Add report fields:

```
production_compile_cache_hit
production_compile_ms
production_topology_hash
```

Local integration order

The local bridge must compile the Production Mode workflow before all of these operations:

- model-stack extraction
- active-next-profile generation
- workflow hash used for warmup selection
- remote Modal submission
- local required-node analysis

Exact order:

1. Read the current graph metadata.
2. Apply temporary native bypass modes.
3. Serialize the API prompt.
4. Restore original modes.
5. Normalize production options.
6. Compile the serialized API prompt.
7. Produce the production report.
8. Use the compiled workflow for model-stack extraction.
9. Use the compiled workflow for active-next-profile.
10. Submit the compiled workflow and production options to Modal.

Do not derive warmup/model information from the original unpruned workflow. Otherwise an unused test branch could cause the wrong model to preload.

Remote integration order in `comfyapp.py`

Both `run_prompt()` and `run_prompt_stream()` must defensively compile the workflow again.

The production compilation must occur immediately after request identity/trace initialization and before:

- `_cold_unet_early_actual_load()`
- `_handle_custom_node_sync_and_dependency_policy()`
- `_prompt_async_preload()`
- `_prompt_async_actual_load()`
- `extract_requested_model_stack()`
- missing-node checks
- `_execute_in_process()`
- subprocess prompt submission

Use one local variable:

```
execution_workflow
```

Rules:

```
execution_workflow = workflow
if production["enabled"]:
    execution_workflow, production_report = compile_production_workflow(...)
```

After this point in each request method, every operation listed above must receive `execution_workflow`, not the original `workflow`.

Attach the report to the final result:

```
result["_production"] = production_report
```

Add one compact log:

```
[production.compile] enabled=1 original_nodes=31 kept_nodes=24 removed_nodes=7
bypassed_nodes=1 outputs=1 direct_outputs=1 cache_hit=1 compile_ms=0.4
hash=<hash>
```

When disabled:

- do not compile
- do not add production-only output restrictions
- do not change the original workflow object

Direct in-memory production output

Purpose

Avoid this existing path for compatible production outputs:

```
IMAGE
→ PNG encoding by output node
→ output/temp file write
→ file discovery
→ file read
→ PNG decode
→ WebP/JPEG conversion
→ base64 encoding
```

Use:

```
IMAGE tensor
→ final requested encoder
→ bytes
→ base64
```

New internal node class

Add a remote-only node class inside `comfyapp.py`:

```
class ComfyModalProductionOutput
```

Register it in `nodes.NODE_CLASS_MAPPINGS` during `_start_in_process_backend()` after built-in and custom nodes have initialized.

Required interface:

```
CATEGORY = "_for_internal_use/ComfyModal"
OUTPUT_NODE = True
FUNCTION = "encode"
RETURN_TYPES = ()
```

Inputs:

```
@classmethod
def INPUT_TYPES(cls):
    return {
        "required": {
            "images": ("IMAGE",),
        },
        "hidden": {
            "unique_id": "UNIQUE_ID",
        },
    }
```

Do not expose this node in the local add-node menu. It is remote-only and injected by the compiler.

Request context

Before executor execution, store request-local production settings on the ComfyAPI instance:

```
self._active_production_request = {
    "enabled": True,
    "prompt_id": prompt_id,
    "output_format": ...,
    "quality": ...,
    "webp_lossless_compression": ...,
    "metadata_mode": ...,
}
```

Clear it in a `finally` block after execution.

Create:

```
self._production_output_registry = {}
```

Registry shape:

```
{
  prompt_id: {
    node_id: [
      {
        "filename": str,
        "bytes": bytes,
        "mime_type": str,
        "width": int,
        "height": int,
        "output_index": int,
      }
    ]
  }
}
```

Key everything by prompt ID and node ID even though current Modal concurrency is one input.

Node execution

Inside `ComfyModalProductionOutput.encode()`:

1. Use `comfy_execution.utils.get_executing_context()` to obtain prompt ID and node ID.
2. Verify an active production request exists.
3. Accept IMAGE tensors shaped `[B, H, W, C]`.
4. Clamp values to `[0, 1]`.
5. Multiply by 255.

6. Convert to `uint8`.
7. Move to CPU once.
8. Encode each batch image directly into the requested final format:
9. original → PNG
10. webp_lossless → lossless WebP using configured method
11. webp_lossy → lossy WebP using configured quality and method
12. jpeg → RGB JPEG using configured quality
13. Do not first encode to PNG when WebP or JPEG is requested.
14. Do not write output or temp files.
15. Store encoded bytes in `_production_output_registry`.
16. Return a minimal output-node response that lets PromptExecutor consider the node successfully executed without putting raw bytes into normal history JSON.

Filename format:

```
production_<prompt_id_first8>_<node_id>_<batch_index>.<extension>
```

Direct-output rewrite eligibility

The compiler may rewrite a selected Production Output node to `ComfyModalProductionOutput` only when:

1. Production direct output is enabled.
2. The selected node has an input named `images`.
3. `inputs["images"]` is a valid node connection.
4. The selected class type is one of:
5. `SaveImage`
6. `PreviewImage`
7. `SaveImageWithMetaData`

Rewrite only:

```
{
  "class_type": "ComfyModalProductionOutput",
  "inputs": {
    "images": original_spec["inputs"]["images"]
  }
}
```

Do not preserve filename prefix or development metadata fields.

For `SaveImageWithMetaData`, Production Mode intentionally follows `metadata_mode`:

- none: no embedded metadata
- minimal: only prompt ID, node ID, width, height, and output format
- full: do not use the direct sink; retain the original node and fallback file path

Default metadata mode is `none`.

For unsupported output classes:

- do not rewrite
- retain the original output node
- use strict filtered history collection

For video or animated outputs:

- never use the direct image sink
- retain existing output behavior

Sampler latent preview suppression

The current progress hook ignores the preview image payload, but that alone does not prevent ComfyUI from generating the latent preview.

Add a context manager:

```
@contextlib.contextmanager
def _production_preview_mode(self, enabled: bool):
```

Behavior:

1. If disabled, yield without changes.
2. Acquire a module-level preview-method lock.
3. Import `latent_preview`.
4. Save the current preview method.
5. Call:

```
latent_preview.set_preview_method("none")
```

1. Execute the prompt.
2. In `finally`, restore the saved/default preview method.
3. Release the lock.

Wrap only the actual PromptExecutor execution, not backend initialization.

Warmup execution with `collect_outputs=False` must retain existing behavior unless Production Mode was explicitly passed.

Do not globally change the user's normal local ComfyUI preview setting.

Progress event throttling

The current `_progress_hook()` sends every progress update.

Add per-request state:

```
self._active_progress_policy = {
    "production_enabled": bool,
    "minimum_interval_s": 0.5,
    "last_emitted_at": 0.0,
    "last_emitted_value": -1,
}
```

In the global progress hook:

1. Always construct the progress data.
2. Always call `_note_progress_event()` directly so internal timing remains accurate.
3. When Production Mode is disabled:
4. preserve current event behavior.
5. When Production Mode is enabled:
6. emit the first progress event
7. emit the final progress event
8. emit intermediate events only if at least 500ms passed since the previous emitted event
9. never include preview image bytes
10. Reset policy state after every request.

In `_on_sync()`:

- Continue internal node timing for `executing` events.
- In Production Mode, do not forward per-node `executing` events into `_prog_queue`.
- Forward:
 - `execution_start`
 - throttled progress

- execution_error
- final result/status
- Preserve all existing events outside Production Mode.

Add result telemetry:

```
progress_events_received
progress_events_forwarded
progress_events_suppressed
```

Quiet execution logs

Scope

Suppress noisy logs only while the graph itself executes.

Do not suppress:

- restore logs
- active model-read logs
- model-load stall logs
- fatal errors
- final compact timing summary
- output collection errors
- Python tracebacks on failure

Implementation

Create:

```
class _ProductionLogCapture
```

It must:

1. Save original stdout and stderr.
2. Maintain a ring buffer of the last 200 complete lines.
3. Count all captured lines.
4. Temporarily route stdout and stderr through the ring buffer.
5. Temporarily update root `logging.StreamHandler` instances whose stream is the original stdout or stderr so standard Python logging is captured too.
6. Restore all streams and handlers in `finally`.
7. On successful execution:
8. discard captured lines

9. print one line:

```
[production.logs] suppressed_lines=<count>
```

1. On execution exception:

2. replay the last 200 captured lines

3. print:

```
[production.logs] replayed_lines=<count> reason=execution_failure
```

- re-raise the original exception

Apply the capture around:

```
self._executor.execute(...)
```

Do not apply it around preflight, restore, model preload, output collection, or final summary generation.

When Production Mode is disabled, do not construct the capture.

Validation and execution-plan caching

The existing `_compute_workflow_struct_hash()` and `_workflow_exec_cache` remain, but update their production behavior.

New execution cache key

When Production Mode is enabled:

```
_wf_cache_key = (  
    f"wf_exec:production:v1:"  
    f"{production_topology_hash}:"  
    f"{compiled_workflow_struct_hash}"  
)
```

When disabled, preserve the existing key.

Cache contents

Store:

```
{
  "outputs_to_execute": list,
  "node_errors": dict,
  "production_output_node_ids": list,
  "compiled_node_ids": list,
}
```

Cache validation

On cache hit, verify:

- every cached output ID still exists
- every cached compiled node ID still exists
- current normalized Production Output IDs match cached IDs

If not, invalidate and run full `execution.validate_prompt()`.

Bound `_workflow_exec_cache` to 32 entries using LRU behavior.

Do not persist it.

Do not skip validation on the first occurrence of a topology.

Production output collection

Update:

```
_collect_in_process_outputs()
```

Add optional production filtering from `modal_options`.

Collection priority in Production Mode

1. Read `_production_output_registry[prompt_id]`.
2. If registry contains entries:
3. package only those entries
4. do not inspect executor history
5. do not inspect PromptQueue history
6. do not scan output directory
7. do not scan temp directory
8. Otherwise inspect `executor.history_result`, but only for selected Production Output node IDs.

9. If required selected outputs are not found, inspect PromptQueue history, again only for selected node IDs.
10. If still not found:
11. fail with a production-output error
12. do not scan directories

Exact error:

```
Production output collection failed. Selected output node(s) <ids> produced no collectible in-memory or history output.
```

Directory scanning remains unchanged outside Production Mode.

Strict output-node filtering

When Production Mode is enabled:

- ignore history outputs from every node not in `output_node_ids`
- do not include `a_images`, `b_images`, comparer outputs, or temp outputs unless their node is explicitly selected as a Production Output
- preserve all output keys from selected nodes
- support multiple selected production output nodes
- preserve batch image order

Update subprocess `_collect_outputs()` with the same output-node filtering. Direct in-memory output is not available for subprocess mode.

Fallback conversion deduplication

The current collector may schedule separate conversions for identical files.

Apply deduplication in both Production and normal mode because it does not change logical output behavior.

Before scheduling `_convert_image_bytes()`:

1. Compute:

```
conversion_key = (  
    sha256(raw_bytes).hexdigest(),  
    output_format,  
    quality,
```

```
webp_lossless_compression,  
)
```

1. Group pending conversion items by this key.
2. Submit one conversion future per unique key.
3. Reuse the converted byte result for every logical output item in that group.
4. Base64-encode each unique converted byte string once.
5. Reuse the base64 string for all identical logical entries.
6. Preserve filenames, node IDs, output keys, and ordering for every logical output.

Add telemetry:

```
conversion_logical_inputs  
conversion_unique_inputs  
conversion_reused_count  
base64_unique_inputs  
base64_reused_count
```

Do not collapse logical outputs outside Production Mode. Only reuse computation.

In Production Mode, unselected logical outputs are already absent.

Duplicate-work analysis

Run this analysis during production compilation.

Do not automatically rewrite compute nodes in this implementation.

Report:

Duplicate output sinks

Two output nodes are duplicates when:

- both receive the same image connection
- both are output-capable nodes

Report their IDs and shared source connection.

Duplicate VAE decodes

Two nodes are potential duplicate VAE decodes when:

- class types are equal
- class type contains `VAEDecode`

- their latent/sample input connections are identical
- their VAE input connections are identical
- all remaining scalar inputs are equal

Report only.

Duplicate CLIP encodes

Two nodes are potential duplicate CLIP encodes when:

- class types are equal
- class type contains `CLIPTextEncode`
- their CLIP input connections are identical
- their text inputs are equal

Report only.

Report shape:

```
{
  "duplicate_output_groups": [...],
  "duplicate_vae_decode_groups": [...],
  "duplicate_clip_encode_groups": [...],
}
```

Show warnings in the sidebar production summary but do not block execution.

Production preflight fast path

The current request path still performs custom-node synchronization and dependency validation.

Do not remove safety validation.

Add a production-only skip for redundant custom-node filesystem synchronization.

During restore or the first successful request, store:

```
self._last_custom_node_sync_state_hash
```

Compute it from the existing cheap custom-node volume state, not the full source-content fingerprint.

For a Production Mode request:

1. Compute the current cheap state hash.

2. If it matches the saved hash:

3. skip `_sync_custom_nodes_from_volume()`

4. report `custom_node_sync_skipped=True`

5. report reason `production_state_unchanged`

6. Still run the cached dependency-validation path.

7. Still check that every class type in the compiled workflow is registered.

8. If state changed:

9. run the full current sync path

10. update the hash

Do not scan the entire model volume for this fast path.

Do not persist the hash to the Modal model volume.

Output and timing telemetry

Attach:

```
result["_production"] = {
    "enabled": True,
    "schema_version": 1,
    "original_node_count": ...,
    "compiled_node_count": ...,
    "removed_node_count": ...,
    "bypassed_node_count": ...,
    "production_output_count": ...,
    "direct_output_count": ...,
    "compile_cache_hit": ...,
    "compile_ms": ...,
    "sampler_previews_disabled": True,
    "quiet_logs_enabled": True,
    "suppressed_log_lines": ...,
    "progress_events_received": ...,
    "progress_events_forwarded": ...,
    "progress_events_suppressed": ...,
    "output_collection_mode": "direct_memory" | "strict_history",
    "filesystem_scan_skipped": True,
    "conversion_logical_inputs": ...,
    "conversion_unique_inputs": ...,
    "conversion_reused_count": ...,
```

```
"duplicate_analysis": ...  
}
```

Add production timings to `trace_summary["derived_ms"]`:

```
production_compile_ms  
production_output_encode_ms  
production_output_package_ms
```

Print one final compact summary:

```
[production.summary] original_nodes=31 kept_nodes=24 removed_nodes=7  
bypassed_nodes=1 outputs=1 direct_outputs=1 compile_ms=0.4 encode_ms=84.2  
files_read=0 filesystem_scan=0 conversion_reused=0 logs_suppressed=418  
progress_suppressed=7
```

Compatibility rules

Production Mode off

Must preserve:

- full workflow
- all outputs
- rgthree `a_images` and `b_images`
- directory scanning
- current progress events
- current logging
- current preview behavior
- current conversion behavior except safe conversion-result reuse
- current return modes

Warmup execution

When `_execute_in_process(..., collect_outputs=False)` is called:

- do not require Production Output IDs
- do not apply dead-branch compilation unless explicitly passed a compiled warmup workflow
- do not use direct output nodes
- do not collect outputs

Scheduler test mode

Force Production Mode off for scheduler-test execution.

Log:

```
[production.compile] enabled=0 reason=scheduler_test
```

Subprocess backend

Support:

- dead-branch compilation
- native bypass serialization performed locally
- selected output filtering
- quiet remote payload behavior where possible

Do not inject `ComfyModalProductionOutput` into subprocess workflows.

Use the original selected output nodes and strict filtered HTTP output collection.

Videos

Do not route videos through the direct image sink.

Multiple production outputs

Supported.

Retain every selected root and all of its ancestors.

Invalid production metadata

Block before model preload.

No filesystem behavior changes outside Production Mode

Do not delete or relocate output/temp folders.

Do not change the current output folder configuration.

Exact implementation sequence

Implement in this order:

1. Add `production_workflow.py` with pure compiler and tests.
2. Add topology cache and duplicate-work analysis.
3. Add local frontend node properties and context-menu actions.
4. Add the Simulate Production checkbox and first-enable output selector.

5. Apply native bypass during API-prompt serialization.
6. Compile locally before model-stack extraction and active profile creation.
7. Add remote production-option normalization.
8. Compile remotely at the beginning of `run_prompt()` and `run_prompt_stream()`.
9. Pass the compiled workflow through all preload, validation, execution, and diagnostic paths.
10. Add sampler preview suppression.
11. Add progress throttling without breaking internal timing.
12. Add quiet execution-log capture.
13. Add remote-only `ComfyModalProductionOutput`.
14. Add direct-output rewrite and registry collection.
15. Add strict selected-node output filtering.
16. Add fallback conversion and base64 deduplication.
17. Add production preflight sync fast path.
18. Add telemetry and sidebar summary.
19. Add all tests.
20. Bump `COMFYAPP_VERSION` from `2.16.5` to `2.16.6`.

Do not reorder this sequence.

Automated tests

Pure compiler tests

Add tests for:

1. One selected output with two dead preview branches.
2. Multiple selected outputs.
3. Shared ancestors retained once.
4. Disconnected model-loader branch removed.
5. Scalar inputs preserved.
6. Input connection detection does not treat arbitrary lists as graph edges.
7. Missing Production Output ID rejected.
8. Output also marked bypass rejected.
9. Bypass ID remaining after frontend serialization rejected.
10. Stable topology hash when only prompt text changes.
11. Stable topology hash when only seed changes.
12. Changed topology hash when a graph connection changes.
13. Changed topology hash when production outputs change.
14. Cache hit reconstructs current scalar values.
15. LRU never exceeds 32 entries.

Direct-output tests

1. PNG encoding from IMAGE tensor.
2. Lossless WebP encoding.
3. Lossy WebP encoding.
4. JPEG encoding with RGB conversion.
5. Batch order preserved.

6. No output or temp file is written.
7. Registry is keyed by prompt and node ID.
8. Registry clears after success.
9. Registry clears after failure.
10. Unsupported output class remains unchanged.
11. Video output remains unchanged.
12. Full metadata mode disables direct rewrite.

Output collection tests

1. Production registry output skips all filesystem scans.
2. Strict history mode reads only selected node IDs.
3. Unselected `a_images` and `b_images` are ignored.
4. Missing selected output causes the exact production error.
5. Production Mode off preserves all current collector sources.
6. Duplicate raw files convert once but retain logical entries.
7. Base64 encoding occurs once per unique converted result.
8. Output order remains stable.

Preview and progress tests

1. Preview method becomes none during production execution.
2. Previous preview method restores after success.
3. Previous preview method restores after exception.
4. First progress event forwarded.
5. Final progress event forwarded.
6. Intermediate events throttled to 500ms.
7. Internal sampler timing still receives all events.
8. Production stream does not forward per-node executing events.
9. Non-production stream remains unchanged.

Logging tests

1. Successful production execution suppresses noisy stdout/stderr.
2. Successful execution prints one suppression count.
3. Failed execution replays the last 200 lines.
4. Streams and logging handlers restore after success.
5. Streams and logging handlers restore after failure.
6. Non-production logging remains unchanged.

Request-path tests

1. Remote compilation occurs before cold UNET early load.
2. Compiled workflow is used for dependency policy.
3. Compiled workflow is used for async preload.
4. Compiled workflow is used for actual-load extraction.
5. Compiled workflow is used for execution.
6. Compiled workflow is used for warmup/model-stack diagnostics.
7. Production Mode does not modify the input workflow object.

8. Scheduler tests disable Production Mode.
9. Production sync fast path skips only unchanged custom-node sync.
10. Dependency validation still runs through its existing cache.
11. Missing node classes still block before model work.
12. Production feature creates no model-volume writes or commits.

Frontend tests

1. Checkbox defaults off.
2. Checkbox state persists in workflow metadata.
3. First enable with zero outputs opens output selection.
4. Cancel leaves Production Mode off.
5. At least one selected output enables Production Mode.
6. Context-menu marks persist.
7. Bypass serialization restores original node modes.
8. Failed serialization restores original node modes.
9. Saved workflow node modes are never changed by production submission.
10. Production summary displays compiler counts.
11. Invalid plan blocks queueing.
12. Production options contain no secrets or image data.

Manual verification workflow

Use a workflow containing:

- one final SaveImage or SaveImageWithMetaData output
- one PreviewImage branch
- one rgthree comparer producing `a_images` and `b_images`
- one debug SaveImage branch
- one inline node marked Bypass in Production
- the current Z-Image UNET, CLIP, and VAE stack

Verify with Production Mode off:

- all existing output files remain
- comparer outputs remain
- sampler progress remains
- logs remain unchanged

Verify with Production Mode on:

- only selected final output is returned
- preview/comparer/debug branches do not execute
- inline bypass node is absent from submitted API workflow
- sampler latent preview generation is disabled
- final image is encoded directly in memory for supported output nodes
- output and temp directories receive no file from the direct production sink
- no output/temp directory scan occurs

- logs are strongly reduced
- progress still moves but is throttled
- exact image dimensions and generation settings are unchanged
- result contains `_production` diagnostics

Benchmark plan

Do not deploy or run paid benchmarks automatically.

After local tests pass, report commands for:

A. Existing behavior

- Production Mode off
- 3 cold runs

B. Production compiler

- Production Mode on
- same seed, prompt, model, steps, scheduler, sampler, dimensions, and output format
- 5 cold runs

Compare:

```
known_nonoverlap
remote_visible
pre_sampler
sampler
post_sampler
output_collection_total_ms
production_output_encode_ms
file_read_total_ms
return_packaging_ms
files_read
files_returned
bytes_read
bytes_returned
log line count
progress event count
```

Image output must be visually and dimensionally equivalent. Lossless formats must remain lossless.

Prohibited changes

Do not:

- install ComfyUI-to-Python-Extension
- replace PromptExecutor
- implement direct manual calls to arbitrary node methods
- alter model-loading policies
- enable direct UNET warmup
- enable UNET CPU preload
- enable safetensors read_bytes
- enable FUSE governor
- change restore-background UNET behavior
- change direct CLIP policy
- disable CacheDiT
- change sampler, scheduler, steps, CFG, denoise, precision, or dimensions
- globally enable first-image-only return mode
- remove comparer support outside Production Mode
- delete temp outputs outside Production Mode
- change Modal scaledown_window
- edit the deploy/run batch files
- create model-volume telemetry writes
- call `vol.commit()` for production compiler caches or production metrics
- make git commits unless explicitly requested

Validation commands

Run:

```
python -m py_compile comfyapp.py
python -m py_compile production_workflow.py
python -m pytest tests/test_production_workflow.py -v
python -m pytest tests/test_comfyapp_runtime_state.py -v
```

Also run existing frontend tests using the repository's current frontend test command.

Do not run:

```
modal deploy
image generation
paid cold benchmarks
```

Final report

Return:

1. Files changed.
2. `COMFYAPP_VERSION` before and after.
3. Exact frontend metadata schema.
4. Exact `modal_options.production` schema.
5. Dead-branch compiler behavior.
6. Native bypass implementation.
7. Direct-output sink behavior.
8. Preview suppression implementation.
9. Progress throttling implementation.
10. Log suppression implementation.
11. Validation and topology-cache behavior.
12. Output filtering and conversion deduplication behavior.
13. Tests added.
14. Commands run and results.
15. Known compatibility limitations.
16. Whether deployment is required.
17. Suggested A/B commands, without running them.
18. Explicit confirmation that all prohibited areas were left unchanged.